

Week 6 – CI/CD, DevOps Automation & Observability

Part A

1. Explain the difference between Continuous Integration, Continuous Delivery and Continuous Deployment.

Continuous Integration (CI) means automatically building and testing the application whenever code changes. The purpose is to find integration and build problems quickly instead of discovering them later.

Continuous Delivery extends CI by automatically preparing a successful build for release. The application is packaged and made ready to deploy, but a person may still need to approve or trigger the release.

Continuous Deployment goes one step further by automatically deploying a successful build to an environment. In my Week 6 project, GitHub Actions performs continuous deployment. When code is pushed to the main branch, the workflow builds the backend Docker image, pushes it to Docker Hub, logs into Rahti, updates the backend Deployment using `oc set image`, and waits for the rollout to complete.

2. Describe the structure of a GitHub Actions workflow (triggers, jobs, steps). What would a minimal pipeline for the app built in Week 4/5 need to do?

A GitHub Actions workflow is a YAML file stored under `.github/workflows/`. It contains triggers, jobs and steps.

The trigger specifies when the workflow starts. In my project, the workflow is triggered when code is pushed to the main branch. A job is a group of tasks executed on a GitHub-hosted runner. My workflow has a build-and-deploy job running on `ubuntu-latest`.

Steps are the individual actions or commands executed by the job. My pipeline checks out the repository, logs into Docker Hub using GitHub Secrets, builds the backend image, pushes the image, installs the OpenShift CLI, logs into Rahti, updates the backend Deployment and waits for the rollout.

For my Week 4/5 application, the pipeline needs to build the backend image, push it to a registry, authenticate to Rahti, update the Deployment to use the new image, and verify that the deployment successfully rolls out.

3. Compare the three deployment-automation patterns discussed (imperative CLI step, ImageStream trigger, GitOps). What are the security trade-offs of each, particularly around who holds cluster credentials?

An imperative CLI step has the CI pipeline directly connect to the cluster and run commands such as `oc apply` or `oc set image`. This is the approach used in Week 6. It is simple, but GitHub Actions needs a Rahti login token, so the CI system has cluster credentials.

An ImageStream trigger allows the cluster to react when a new image is pushed to a registry. The pipeline can mainly build and push the image while OpenShift handles the rollout, reducing the amount of cluster access required by CI.

GitOps stores desired deployment configuration in Git and uses a controller such as Argo CD to reconcile the cluster. This can reduce the need for cluster credentials in GitHub Actions because the deployment controller handles cluster access.

The main security difference is where cluster credentials are stored and used. In the imperative approach, the Rahti token is stored as a GitHub repository secret and used by the workflow. The token should never be written directly into the workflow or committed to Git.

4. What are the three pillars of observability (metrics, logs, traces)? Give an example of a question each one is best suited to answer.

The three pillars are metrics, logs and traces.

Metrics are numerical measurements collected over time, such as CPU usage, memory usage, request rates and error rates. A useful question is: Has the application's error rate increased over the last hour?

Logs are timestamped events generated by an application or system. They provide detailed information about what happened. In my backend, request logs contain the request path, HTTP method, response status and response duration. A useful question is: Which request failed and what response did the backend return?

Traces follow an individual request through different components. They are useful for multi-service applications. A useful question is: Which service in the frontend-to-backend-to-database request caused a delay?

5. Why should secrets (registry tokens, cluster login tokens) never be hard-coded into a CI/CD pipeline file? How does GitHub Actions handle this instead?

Secrets such as Docker Hub access tokens and Rahti/OpenShift login tokens should never be hard-coded into a workflow because the workflow is stored in Git. Anyone with inappropriate access to the repository or its history could potentially obtain the credentials.

GitHub Actions provides repository secrets for this purpose. In my project I created DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, RAHTI_SERVER and RAHTI_TOKEN. The workflow accesses them using expressions such as `${{ secrets.DOCKERHUB_TOKEN }}` and `${{ secrets.RAHTI_TOKEN }}`.

The actual token values are therefore not written into the workflow file. This follows the same principle used for Kubernetes/OpenShift Secrets: credentials should be kept separate from application and configuration code and should not be committed to Git.

Part B — Hands-On: Automate & Observe the App

1. CI/CD Pipeline

I created a GitHub Actions workflow at `.github/workflows/backend-cicd.yml`. The workflow is triggered whenever code is pushed to the main branch.

The pipeline checks out the repository, logs into Docker Hub using GitHub repository secrets, builds the backend Docker image, tags it with the Git commit SHA, pushes it to Docker Hub, installs the OpenShift CLI, logs into Rahti, updates the backend Deployment using `oc set image`, and waits for the rollout to complete.

The workflow uses the following image build command:

```
docker build -t upeksha94/lemp-backend:${{ github.sha }} -f backend/Dockerfile .
```

It then pushes the image and deploys the same commit-specific image to Rahti.

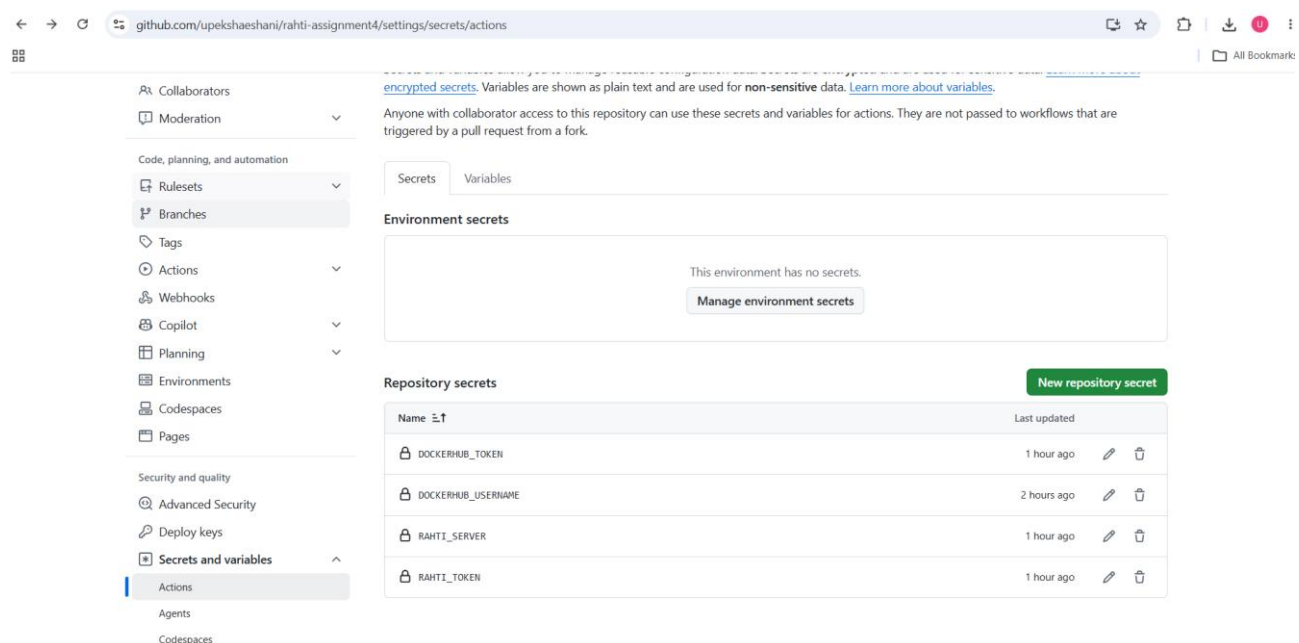
The credentials are stored as GitHub repository secrets:

DOCKERHUB_USERNAME

DOCKERHUB_TOKEN

RAHTI_SERVER

RAHTI_TOKEN



2. Successful CI/CD Run

The workflow was committed and pushed to the repository. The successful workflow run was triggered by commit `8fadaff`.

The deployed image was: `upeksha94/lemp-`

`backend:8fadaffa9e7b23634dc31e788983e759afca2929`

I verified it with:

```
oc.exe get deployment backend -o jsonpath='{.spec.template.spec.containers[0].image}'
```

The result confirmed that the Rahti Deployment was using the image tagged with the Git commit SHA. This demonstrates that the pipeline performed a real deployment change rather than only building an image.

The screenshot shows a GitHub Actions workflow run for the repository 'upekshaeshani / rahti-assignment4'. The workflow is named 'Add Week 6 backend CI/CD pipeline #1' and has a status of 'Success'. It was triggered by a push to the 'main' branch. The total duration of the workflow is 49s. The workflow file is 'backend-cicd.yml' and it runs on a push event. The workflow consists of a single job named 'build-and-deploy' which has a duration of 46s. The workflow is located in the 'Backend CI/CD' directory.

3. Evidence of Real Deployment

I checked the backend pods in Rahti

```
C:\Users\USER\oc>oc.exe get pods -l app=backend
NAME                                READY    STATUS    RESTARTS    AGE
backend-6dc9fdc89d-c55cz            1/1      Running   0            2m59s
```

I also verified the Deployment image and confirmed that it was the commit-specific image produced by GitHub Actions.

4. Health Checks

I added two health endpoints to the Flask backend.

The liveness endpoint is /healthz and returns a status of ok. It is used by OpenShift to determine whether the backend container is alive.

The readiness endpoint is /readyz. It attempts to connect to MySQL. If the connection succeeds, it returns HTTP 200. If the database connection fails, it returns HTTP 503.

This makes the readiness check dependent on an important backend dependency.

5. Probe Configuration

The backend Deployment contains these probes:

```
livenessProbe:
  httpGet:
    path: /healthz    port:
    8000
  initialDelaySeconds: 10
  periodSeconds: 15
  readinessProbe: httpGet:
    path: /readyz    port:
    8000
  initialDelaySeconds: 5
  periodSeconds: 10
```

I verified the configuration with `oc describe pod`. The output showed the liveness probe using `/healthz` and the readiness probe using `/readyz`.

```
Liveness:  http-get http://:8000/healthz delay=10s timeout=1s period=15s #success=1 #
Readiness: http-get http://:8000/readyz delay=5s timeout=1s period=10s #success=1 #fa
```

6. Testing the Readiness Probe

I deliberately changed the readiness probe to an endpoint that does not exist:

The resulting pod remained Running but was not Ready:

This demonstrated that a readiness failure does not necessarily restart the container. Instead, the pod becomes unavailable for normal traffic.

I then restored the correct endpoint

After the rollout, the backend returned to:

```
backend-6dc9fdc89d-c55cz 1/1 Running
```

This demonstrated recovery of the readiness check.

```
C:\Users\USER\oc>oc.exe patch deployment backend --type=json -p="[{\"op\":\"replace\", \"path\":\"/spec/template/spec/containers/0/readinessProbe/httpGet/path\", \"value\":\"/does-not-exist\"}]"
deployment.apps/backend patched

C:\Users\USER\oc>oc.exe get pods -l app=backend
NAME                                READY   STATUS    RESTARTS   AGE
backend-6dc9fdc89d-c55cz            1/1     Running   0           15m
backend-b89bf9854-4cwv9             0/1     Running   0           34s

C:\Users\USER\oc>oc.exe patch deployment backend --type=json -p="[{\"op\":\"replace\", \"path\":\"/spec/template/spec/containers/0/readinessProbe/httpGet/path\", \"value\":\"/readyz\"}]"
deployment.apps/backend patched

C:\Users\USER\oc>oc.exe get pods -l app=backend
NAME                                READY   STATUS    RESTARTS   AGE
backend-6dc9fdc89d-c55cz            1/1     Running   0           16m
```

7. Structured Backend Logging

I added request logging to the Flask backend. The log entries contain the request path, HTTP method, response status and response duration.

These entries make it easier to understand which requests are being handled and how long they take.

```
2026-10-03 05:48:08,384 INFO request path=/readyz method=GET status=200 duration_ms=17.39
2026-10-03 05:48:17,469 INFO request path=/healthz method=GET status=200 duration_ms=0.11
2026-10-03 05:48:18,385 INFO request path=/readyz method=GET status=200 duration_ms=17.73
2026-10-03 05:48:28,384 INFO request path=/readyz method=GET status=200 duration_ms=16.32
2026-10-03 05:48:32,469 INFO request path=/healthz method=GET status=200 duration_ms=0.14
2026-10-03 05:48:38,384 INFO request path=/readyz method=GET status=200 duration_ms=16.43
2026-10-03 05:48:47,469 INFO request path=/healthz method=GET status=200 duration_ms=0.12
```

8. Retrieving Logs from Rahti

I verified that the application logs can be retrieved using:

```
oc.exe logs deployment/backend --tail=20
```

The output contained repeated successful health-check requests,

The requests demonstrate that the OpenShift health probes are reaching the backend and receiving successful responses.

```
C:\Users\USER\oc>oc.exe logs deployment/backend --tail=20
2026-10-03 07:00:03,730 INFO request path=/readyz method=GET status=200 duration_ms=15.47
2026-10-03 07:00:11,751 INFO request path=/healthz method=GET status=200 duration_ms=0.14
2026-10-03 07:00:13,731 INFO request path=/readyz method=GET status=200 duration_ms=16.28
2026-10-03 07:00:23,730 INFO request path=/readyz method=GET status=200 duration_ms=15.59
2026-10-03 07:00:26,751 INFO request path=/healthz method=GET status=200 duration_ms=0.09
2026-10-03 07:00:33,731 INFO request path=/readyz method=GET status=200 duration_ms=16.46
2026-10-03 07:00:41,751 INFO request path=/healthz method=GET status=200 duration_ms=0.09
2026-10-03 07:00:43,731 INFO request path=/readyz method=GET status=200 duration_ms=15.75
2026-10-03 07:00:52,730 INFO request path=/readyz method=GET status=200 duration_ms=15.17
```

9. Example Alert

A useful alert for this application would be triggered if the backend Deployment has no ready pods for several minutes. Another possible alert could trigger if the application has a sustained high rate of HTTP 5xx responses.

I would avoid creating alerts for every individual failed request because temporary failures can occur and excessive notifications can cause alert fatigue.

10. Problems Encountered and Solutions

One problem occurred when I ran an OpenShift command from the wrong directory. The Kubernetes YAML file was in the repository, so I solved this by specifying its full path.

Another problem occurred while testing the readiness probe because of incorrect command-line quoting. I corrected the `--type` argument to `--type=json`, after which the patch worked, and the readiness failure could be demonstrated.

11. Reflection

This week's work changed my deployment process from manually building and deploying the backend to an automated workflow. A push to the main branch now triggers GitHub Actions, which builds and pushes a Docker image and then updates the backend Deployment in Rahti.

Using the Git commit SHA as the Docker image tag makes the deployment traceable to a specific version of the source code.

The health probes provide another layer of reliability. The liveness probe checks whether the application is functioning, while the readiness probe checks whether it is ready to receive traffic. I tested the readiness probe by deliberately changing its path and observed the pod becoming 0/1 while remaining Running.

The structured request logs provide useful operational information such as the request path, status and response duration. Together, the CI/CD pipeline, health checks and logging provide a foundation for automating and observing the application.

build-and-deploy		Search logs			
succeeded 1 hour ago in 46s					
>	✔ Set up job				1s
>	✔ Checkout repository				1s
>	✔ Log in to Docker Hub				1s
>	✔ Build backend image				10s
>	✔ Push backend image				6s
>	✔ Set up OpenShift CLI				2s
>	✔ Log in to Rahti				2s
>	✔ Deploy backend				2s
>	✔ Wait for deployment				17s
>	✔ Post Log in to Docker Hub				1s
>	✔ Post Checkout repository				0s
>	✔ Complete job				0s